

AI-Based Storage Tier Recommendation for Medical Image Repositories using Deep Learning Analytics

Research Gap Analysis and Literature Survey (Papers 6 to 10)

Course Instructor: Dr. Priya V | Submission: 30 July 2026

Name: Subhrojyoti Das

Reg No: 24BIT0194

1. What This Project Is About

Hospitals store a very large number of scans such as CT, MRI and X-ray. Keeping all of them on fast storage is expensive. Moving all of them to cheap storage is also a problem, because cheap storage can take hours to give a file back, and a doctor cannot wait that long.

Amazon S3 offers different storage classes. S3 Standard is fast but costly. Glacier Deep Archive is very cheap but slow. The aim of this project is to use deep learning to decide which storage class each scan should go into.

My teammates have already covered ten papers between them. To avoid repeating their work, I studied five different papers that no one else in the team has taken. My five papers approach the same problem from the storage system side and the hospital infrastructure side. For each paper I have written what the authors did, what data they used, what is good about it, what is missing, and what gap is left that our project can fill.

2. The Five Papers I Studied

#	Title	Source / Publisher	Year
P6	Harmonia: Enhancing Data Placement and Migration in Hybrid Storage Systems via Multi-Agent Reinforcement Learning	arXiv preprint (SAFARI Group, ETH Zurich)	2025 (rev. 2026)
P7	Machine Learning-Driven Intelligent Memory System Design: From On-Chip Caches to Storage	arXiv preprint (ETH Zurich)	2026
P8	Cold-RL: Learning Cache Eviction with Offline Reinforcement Learning for NGINX	arXiv preprint (open-source code)	2025
P9	Kaapana: A Comprehensive Open-Source Platform for Integrating AI in Medical Imaging Research Environments	arXiv preprint (DKFZ, Germany)	2025
P10	Project Imaging-X: A Survey of 1000+ Open-Access Medical Imaging Datasets for Foundation Model Development	arXiv preprint (127 authors)	2026

A note on why all five are preprints

My teammates' papers are from journals such as Springer, MDPI and Nature. All five of my papers are arXiv preprints instead. This is not an accident, and it is worth explaining.

- The guideline asks for recent work (2025 to 2026). Journal review takes 12 to 24 months, so almost nothing published in a journal today is actually from 2025 or 2026 in terms of the research itself.
- Papers P6 and P7 come from the SAFARI research group at ETH Zurich, led by Prof. Onur Mutlu. This is one of the best known computer architecture groups in the world and their work reliably appears later at top conferences such as ISCA, MICRO and HPCA. Paper P9 comes from DKFZ, the German Cancer Research Centre.
- All five are free to download with no paywall and no login, which matters for a student project.

- Before final submission I will search each title again and cite the journal or conference version if one has appeared by then.

3. Research Gap Analysis (Paper by Paper)

3.1 Paper 6 | Harmonia: Enhancing Data Placement and Migration in Hybrid Storage Systems via Multi-Agent Reinforcement Learning

R. Nadig, V. Arulchelvan, R. Bera, T. Shahroodi, G. Singh, A. Kakolyris, I. E. Yuksel, M. Sadrosadati, J. Park and O. Mutlu, *arXiv:2503.20507, 2025 (revised 2026)*.

In one line: *Two AI agents work together to decide where a file should be kept and when it should be moved, but they only watch how the file behaves and never look at what is inside it.*

What the authors did:

Earlier systems used one AI agent to do one job. Harmonia uses two small reinforcement learning agents that talk to each other. The first agent is the placement agent. When a new file arrives it decides which device that file should be written to. The second agent is the migration agent. It watches files that are already stored and decides when a file should be moved to a different device. Because the two agents share what they have learned, they do not fight each other. Both agents are deliberately kept small so that the decision can be made faster than the storage device itself can respond.

Data they used:

Twenty-five real workloads were used. These include SYSTOR17 enterprise office traces (LUN0 to LUN4), RocksDB database traces (ssd-00 to ssd-04), YCSB cloud database traces (YCSB-B to YCSB-F), MLPerf Storage machine learning traces (ResNet50 and CosmoFlow), three traces from large language model work, and five production traces from Meta called Baleen which were kept aside as unseen tests. Testing was done on real hardware: an Intel Optane P4800X SSD, an Intel D3-S4510 SSD, a Seagate ST1000DM010 hard disk and an ADATA SU630 SSD, arranged in two, three and four device combinations.

Advantages:

- It handles both jobs together. Placement decides where a file goes at the start, migration decides when to move it later. Our project needs both, because a scan that is busy today may be forgotten next year.
- The improvement is large. Compared with the previous best method it reduced average waiting time by 29.3 percent on a speed-focused setup and 44.8 percent on a cost-focused setup, and it gave 136.7 percent more operations per second on the cost-focused setup.
- It works with three and four devices, not just two. This matters because Amazon S3 also has four main classes.
- It is extremely light. A decision takes about 240 nanoseconds and both agents together need only 206 KiB of memory. This proves that a learning based tier system does not slow the storage down.
- It was tested on unseen workloads from Meta that it had never trained on, so the results are not just memorised.

Limitations:

- There is nothing medical anywhere in the paper. The workloads are office desktops, databases and machine learning jobs. No scan, no hospital, no patient.
- The agents only see numbers about behaviour: how big the request is, whether it is a read or a write, how often it is accessed. They never see what the file contains.
- The word "cost" in this paper means cheaper hardware, not a cloud bill. There is no rupee or dollar figure anywhere, and no charge for moving a file or pulling it back.

- The settings of the agents were tuned once, offline, by hand. They do not retune themselves.
- There is no idea of a file that must legally be kept for a fixed number of years.

RESEARCH GAP

Harmonia solves the placement and migration problem beautifully, but it is completely blind to meaning. Its agents judge a file only by how it has behaved so far. In a hospital this fails in a specific way. A follow-up CT of a cancer patient and a routine chest X-ray of a healthy person can have exactly the same size, the same age and the same number of past opens. Harmonia would treat them identically. The cancer scan could be sent to deep archive, and when the doctor asks for it at the next appointment it would take hours to come back. The paper also measures cost as cheaper hardware, not as the actual monthly cloud bill, and it has no way to obey the rule that a hospital must keep a scan for a fixed number of years.

How we can improve on this:

We will keep the two-agent design because it fits our problem exactly, and we will make three changes. First, we will add new inputs to what the agents can see: features taken from the image itself using a deep learning model, plus DICOM details such as the scan type, the body part and the patient age. Second, we will replace the reward. Instead of rewarding only speed, we will reward the actual Amazon S3 monthly cost, including the transfer and retrieval charges this paper leaves out. Third, we will make the penalty unequal on purpose. Paying a little extra money is a small mistake; sending a scan a doctor needs into deep archive is a big mistake, and the reward will say so.

3.2 Paper 7 | Machine Learning-Driven Intelligent Memory System Design: From On-Chip Caches to Storage

R. Bera, R. Nadig and O. Mutlu, *arXiv:2603.14583*, 2026.

In one line: *This paper shows that a learning system beats hand-written rules at deciding where data should be kept, but the system it describes looks at only six numbers, and not one of them says what the data actually is.*

What the authors did:

This is an overview paper. It brings together three learning based methods built by the same group. Pythia is a reinforcement learning prefetcher for processor caches. Hermes is a small perceptron that predicts whether data will be found in the cache or not. Sibyl is the one that matters to us: it is a reinforcement learning system that decides which storage device a page of data should be placed on in a mixed storage system. The paper argues that memory and storage systems should tune themselves instead of relying on rules written by engineers.

Data they used:

This being an overview paper, there are no new experiments. Sibyl was originally tested on real machines using fourteen storage traces from Microsoft Research Cambridge, collected from enterprise servers. Sibyl describes each page using exactly six numbers: request size, request type (read or write), access interval, access count, how much free space is left on the fast device, and where the page is currently placed.

Advantages:

- It proves the core idea of our project is sound. Sibyl beat hand-written rules by 19.9 to 45.9 percent and beat earlier learning based methods by 34.1 to 68.8 percent.
- It was tested on real hardware inside the operating system storage layer, not only in a simulator.
- It states its six inputs openly. This is very useful for us, because we can point at that exact list and show what is missing from it.

- It is honest about what Sibyl cannot do. The authors say plainly that Sibyl handles placement but not migration, which is why they later built Harmonia.
- It gives us a clear story for the report: across the whole memory hierarchy, learning beats hand-written rules again and again.

Limitations:

- All six inputs describe system behaviour. Not one describes the content or the purpose of the data.
- The Microsoft Research Cambridge traces come from ordinary enterprise servers. They contain no medical data of any kind.
- There is a cold start problem. A file that has just arrived has no access history, so four of the six inputs are effectively empty and the system is guessing.
- Sibyl decides between storage devices inside one machine. It does not deal with cloud storage classes or cloud pricing.
- Being an overview paper, it presents no new experiment that we can repeat.

RESEARCH GAP

The six-number input list of Sibyl is the research gap of our whole project stated in a single line. Request size, read or write, access interval, access count, free space and current location. Every one of these describes behaviour. Not one describes what the data is. A hospital, however, already knows a great deal about a scan the moment it is created: the modality, the body part, the patient age, whether it is urgent, and what the report says. All of this is free and already stored in the DICOM header, and all of it is thrown away by this approach. The cold start problem makes it worse. A newly created scan has no access history at all, so exactly when the placement decision must be made, the system knows the least.

How we can improve on this:

We will extend the six-number list with two new kinds of input. The first is DICOM metadata, which is already present in every scan and costs nothing to read. The second is a small feature vector produced by a deep learning model that looks at the image itself. This directly fixes the cold start problem, because a brand new scan has no access history but it does have a modality, a body part and an image. Our system will therefore be able to place a scan correctly on the very first day, which Sibyl cannot do.

3.3 Paper 8 | Cold-RL: Learning Cache Eviction with Offline Reinforcement Learning for NGINX

A. Gupta and A. Bhayani, *arXiv:2508.12485*, 2025. Source code released on GitHub under CC-BY 4.0.

In one line: *This paper trains its AI entirely from old log files instead of experimenting on a live system, which is exactly the only way a hospital would ever allow us to train ours.*

What the authors did:

NGINX is a very widely used web server. When its cache becomes full it throws out the least recently used file. The authors replaced that fixed rule with a small neural network called a dueling Deep Q-Network. When space is needed, the network looks at the K least recently used objects and scores each one using four things: how old it is, how big it is, how many times it has been opened, and how long it is allowed to live. It then picks which ones to remove. The whole decision must finish in microseconds.

Data they used:

No live system was ever experimented on. Real NGINX access logs were replayed through a cache simulator, and the model learned from that replay. The reward was simple: if a file that was kept is opened again before it expires, the model earns a point. Three cache sizes were tested, 25 MB, 100 MB and 400 MB.

Advantages:

- The training method is the most useful part for us. Learning happens offline from recorded logs. We can never let a learning agent experiment freely on a real hospital archive, so this is the only workable route.
- The four inputs it uses (age, size, number of opens, time to live) map almost one to one onto information we already have for every scan in a hospital archive.
- The improvement is real. At a 25 MB cache the hit rate rose from 14.36 percent to 35.38 percent, and at 100 MB from 75.30 percent to 86.75 percent.
- The overhead is under 2 percent of CPU, so it does not slow the server down.
- It is honest. At a 400 MB cache the learned method only matched the old rule at about 91.8 percent and gave no benefit. The authors report this instead of hiding it.
- The code is public, so we can actually study a working implementation.

Limitations:

- It is a web cache, not a storage archive. It decides what to delete, not which of four storage classes a file should live in.
- There are only two outcomes: stay in the cache, or be removed. Our project needs four.
- The reward is a simple yes or no. It has no idea that some mistakes are far more costly than others.
- There is no money in the paper at all. Nothing is measured in cost.
- The benefit shrinks as the cache grows. When there is enough space, the learned method stops helping.
- There is no medical data and no image of any kind.

RESEARCH GAP

Cold-RL solves a two-way question: keep this file, or throw it away. Our project has to solve a four-way question across S3 Standard, Infrequent Access, Glacier and Deep Archive. More importantly, the cost of being wrong is completely different. If Cold-RL wrongly throws out a web page, the server fetches it again in a few milliseconds and nobody notices. If our system wrongly sends a scan to Glacier Deep Archive, a doctor may wait twelve hours for an image they need now. A simple yes-or-no reward, which is what this paper uses, cannot express that difference at all. It treats every mistake as equally bad.

How we can improve on this:

We will copy the offline training pipeline almost exactly, because it is the only approach a hospital would permit: take recorded access history, replay it through a simulator, and train from that replay. Then we will make two changes. We will widen the decision from two choices to four S3 storage classes. And we will replace the yes-or-no reward with a reward measured in actual money, where wrongly sending a needed scan to archive is punished far more heavily than simply paying a little extra for faster storage.

3.4 Paper 9 | Kaapana: A Comprehensive Open-Source Platform for Integrating AI in Medical Imaging Research Environments

U. Akunal, M. Bujotzek, S. Denner et al., German Cancer Research Centre (DKFZ), arXiv:2512.09644, 2025. Code at github.com/kaapana/kaapana.

In one line: *A complete, free, working hospital imaging platform that already has a PACS, object storage, a DICOM search index and AI pipelines, and yet has no intelligence at all about where each scan should be kept.*

What the authors did:

The authors built and released a full open-source platform for medical imaging research. It brings data ingestion, patient group selection, processing workflows and result viewing into one place. It is built on Kubernetes with Helm for deployment, Apache Airflow to run workflows as step-by-step job graphs, MinIO for S3-compatible object

storage, OpenSearch to index and search DICOM metadata, and Keycloak for login and access control. It includes a built-in PACS that accepts DICOM, a dataset browser, ready-made AI pipelines such as TotalSegmentator and nnU-Net, and image viewers. Its design principle is to send the algorithm to the data rather than move patient data out of the hospital.

Data they used:

This is infrastructure, so there is no dataset and no accuracy number. The evidence is real deployment. Kaapana is in use in the RACOON national radiology network, the CCE-DART consortium, the German Cancer Consortium (DKTK) and the NeuroRad project.

Advantages:

- It is proof that a platform like this really works in hospitals, not only in a paper.
- Its storage layer, MinIO, speaks the same S3 language as Amazon S3. This means the tier logic we write can be plugged in directly with very little change.
- OpenSearch already indexes every DICOM tag of every scan. That index is precisely the source of features our model needs, and it is already built and running.
- Airflow gives us a natural place to run our tier recommendation as a scheduled background job.
- It is completely free and open source, which suits a student project with no budget.
- The send-the-algorithm-to-the-data design fits privacy rules and would let our model be trained across several hospitals without moving patient data.

Limitations:

- Storage is completely flat. Everything sits in MinIO buckets at one single quality of service. There is no concept of a hot tier or a cold tier anywhere.
- There is no cost awareness at all. The platform never asks what the storage is costing.
- There is no automatic lifecycle. Nothing ever moves a scan to cheaper storage as it gets older.
- The AI pipelines are all diagnostic. They segment organs and find disease. AI is never used to run the infrastructure itself.
- The paper reports no measurement of storage efficiency, and states no limitations section of its own.

RESEARCH GAP

Kaapana is the clearest example of the gap in this entire study, because it already owns every single ingredient our project needs and connects none of them. It has a searchable index of every DICOM tag. It has S3-compatible object storage. It has a workflow engine that can run jobs on a schedule. It has deep learning pipelines running over the images. And yet every scan, from a routine X-ray to an urgent trauma CT, is dropped into the same bucket and left there for ever. The DICOM metadata that OpenSearch has already indexed is exactly what a tier prediction model would need, and it is used only to help a researcher search. The link from what the system already knows to where the scan should be stored simply does not exist.

How we can improve on this:

This gives our project a real home instead of a simulation. We will build our tier recommender as an Airflow job inside Kaapana. It will read the DICOM features it needs from the OpenSearch index that is already there, run our deep learning model, and write the resulting storage class decision out to S3 lifecycle rules. That turns Kaapana from a flat store into a cost-aware, tiered archive, and it means our result is something a hospital could actually install rather than only a set of graphs.

3.5 Paper 10 | Project Imaging-X: A Survey of 1000+ Open-Access Medical Imaging Datasets for Foundation Model Development

Z. Deng and 126 co-authors, *arXiv:2603.27460*, 2026. Portal at github.com/uni-medical/Project-Imaging-X.

In one line: *A catalogue of more than a thousand free medical imaging datasets, which solves half our data problem and proves that the other half cannot be solved off the shelf at all.*

What the authors did:

A very large team catalogued over 1,000 publicly available medical imaging datasets and organised them by imaging type, clinical task, body region, kind of annotation, known limitations, and how easily they can be combined with others. They also propose a method they call the metadata-driven fusion paradigm, which is a way of joining several small datasets that share a modality or a task into one larger and more useful dataset. They released an interactive search portal and a single structured table listing everything.

Data they used:

The datasets themselves are the data. There is no experiment and no model. The output is the catalogue, the portal and the public repository.

Advantages:

- It directly solves our requirement for a free and openly available dataset. Instead of searching one dataset at a time, we can filter a single table.
- It records the known limitations of each dataset, so we can choose one honestly and state its weaknesses in our report.
- The fusion idea lets us combine several small datasets into something large enough to train on.
- It is from 2026, so it is completely current.
- The portal and repository are public and free.

Limitations:

- It is a catalogue, not a method. There is no algorithm, no experiment and no result to compare against.
- Every dataset in it is a diagnostic dataset. They are all built to train models that find disease.
- Not a single dataset records storage or access behaviour.
- The authors describe the whole landscape as fragmented, modest in scale, and unevenly spread across body parts and imaging types.
- Many public datasets have been converted to PNG or JPEG, which destroys the DICOM header that our features depend on.
- The authors note that curation is limited by the need for clinical expertise and by strict privacy rules.

RESEARCH GAP

This paper is the strongest evidence we have that our data problem is real. More than a thousand medical imaging datasets are freely available, and not one of them records who opened which scan and when. That record lives only inside hospital PACS audit logs, it is protected patient information, and it is never released publicly. So the very thing our model must learn to predict, namely whether a scan will be needed again, cannot be downloaded from anywhere. It has to be derived. On top of this, many public datasets have been stripped down to plain PNG or JPEG images, which throws away the DICOM header that half of our input features come from.

How we can improve on this:

We will use the catalogue to pick a dataset that still keeps its DICOM headers and that has several scans per patient over time. From those timestamps we will build a proxy label. The reasoning is simple: when a patient comes back for a follow-up scan, the radiologist almost always pulls up the earlier scan to compare. So a later scan for the same patient is good evidence that the earlier one was retrieved. We will state clearly in the report that this is a proxy and not a true access log, explain that it will miss retrievals that happen without a new scan, and test how sensitive our results are to that assumption. Being open about this is stronger than pretending we have real access logs.

4. Overall Research Gap

Putting all five papers together, this is what I found:

- **Paper 6 (Harmonia)** can place files and move them between tiers very well, and it proves the overhead is tiny, but it judges a file only by how it has behaved and never by what it contains.
- **Paper 7 (Sibyl)** proves that learning beats hand-written rules for storage placement, but it uses only six inputs and every one of them is about system behaviour, so a brand new scan cannot be placed intelligently at all.
- **Paper 8 (Cold-RL)** shows how to train such a system safely from recorded logs instead of live experiments, but it only chooses between keep and delete, and it treats every mistake as equally bad.
- **Paper 9 (Kaapana)** already has the DICOM index, the object storage, the workflow engine and the AI pipelines that our project needs, and connects none of them to a storage decision. Every scan sits in the same bucket for ever.
- **Paper 10 (Imaging-X)** proves that over a thousand free medical imaging datasets exist and that not one of them records which scan was opened when, so the label our model needs must be derived rather than downloaded.

So the gap is this. The storage research community has built accurate, extremely fast, learning based tier systems, but it feeds them only block-level behaviour and has never once tested them on medical data. The medical imaging community has built rich deep learning models and complete hospital platforms, but uses them only to help the doctor and never to manage the storage underneath. Nobody has joined the two. No existing work uses deep learning on the medical image itself, together with its DICOM details and its past access record, to predict whether a scan will be needed again and to place it in the correct Amazon S3 storage class, with a penalty that treats a doctor waiting for an archived scan as a far worse mistake than a slightly higher bill. That is what our project will do.

5. Literature Survey Table

Paper	Method	Dataset	Advantages	Limitations	Research Gap
P6 - Nadig et al. (2025), Harmonia, arXiv:2503.20507, SAFARI Group, ETH Zurich	Two small reinforcement learning agents that coordinate with each other. A placement agent decides which device a new file is written to; a migration agent decides when an existing file should be moved. Both are kept small so a decision is faster than the device itself.	25 real workloads: SYSTOR17 enterprise traces (LUN0-4), RocksDB (ssd-00 to 04), YCSB (B-F), MLPerf Storage (ResNet50, CosmoFlow), 3 LLM traces, and 5 unseen Meta production traces (Baleen). Real hardware: Intel Optane P4800X, Intel D3-S4510, Seagate HDD, ADATA SU630, in 2, 3 and 4 device setups.	Handles placement and migration together, which our project needs. 29.3% lower latency on a speed-focused setup and 44.8% on a cost-focused setup; 136.7% more IOPS. Scales to 4 devices, matching the four S3 classes. Only 240 ns per decision and 206 KIB memory. Tested on unseen workloads.	No medical data anywhere; workloads are office, database and ML jobs. Agents see only behaviour numbers, never file content. "Cost" means cheaper hardware, not a cloud bill; no transfer or retrieval charges. Settings tuned once by hand. No legal retention rule.	A follow-up cancer CT and a routine chest X-ray with the same size, age and open count are treated identically, so the important scan can be sent to deep archive and take hours to return. Cost is never measured as an actual monthly bill, and there is no way to keep a scan for the years hospital rules require.
P7 - Bera, Nadig & Mutlu (2026), arXiv:2603.14583, ETH Zurich	Overview paper covering three learning based methods: Pythia (RL prefetcher), Hermes (perceptron cache predictor) and Sibyl (RL data placement for mixed storage). Sibyl describes each page using exactly six numbers and learns which device to place it on.	No new experiments. Sibyl was tested on real machines using 14 storage traces from Microsoft Research Cambridge enterprise servers. Its six inputs are request size, read or write, access interval, access count, free space on the fast device, and current placement.	Proves the core idea works: Sibyl beat hand-written rules by 19.9-45.9% and earlier learning methods by 34.1-68.8%. Tested on real hardware in the OS storage layer, not only simulation. States its six inputs openly. Honest that Sibyl cannot do migration, which led to Harmonia.	All six inputs describe behaviour, none describe content. MSRC traces are ordinary enterprise servers with no medical data. Cold start problem: a new file has no access history. Works between devices in one machine, not cloud storage classes or cloud pricing. Overview paper, so no new experiment to repeat.	The six-input list is the gap in one line. A hospital already knows the modality, body part, patient age, urgency and report text at the moment a scan is created, all free in the DICOM header, and all of it is ignored. The cold start problem makes it worse: exactly when the placement decision must be made, the system knows the least about the file.
P8 - Gupta & Bhayani (2025), Cold-RL, arXiv:2508.12485, code on GitHub	A dueling Deep Q-Network replaces the fixed least-recently-used rule in the NGINX web server. When space is needed it scores the K least recently used objects on age, size, number of opens and time to live, then picks which to remove, all within a microsecond budget.	No live system was experimented on. Real NGINX access logs were replayed through a cache simulator and the model learned offline from that replay. Reward: a kept file that is opened again before expiry earns a point. Cache sizes 25 MB, 100 MB and 400 MB.	Offline training from recorded logs is the only method a hospital would permit, and this proves it works. Its four inputs map almost one to one onto scan attributes we already have. Hit rate rose 14.36% to 35.38% at 25 MB and 75.30% to 86.75% at 100 MB. Under 2% CPU overhead. Honest that at 400 MB it gave no benefit. Code is public.	A web cache, not a storage archive: it decides what to delete, not which of four classes to use. Only two outcomes. Reward is a simple yes or no. No money is measured anywhere. Benefit shrinks as cache grows. No medical data and no images.	The decision here is two-way (keep or delete) and every mistake costs the same: one slow re-fetch that nobody notices. Our decision is four-way, and a wrong move to Glacier Deep Archive means a doctor waits twelve hours for a scan. A yes-or-no reward cannot express that difference at all.
P9 - Akunal et al. (2025), Kaapana, arXiv:2512.09644, DKFZ Germany	A full open-source medical imaging platform. Kubernetes and Helm for deployment, Apache Airflow for workflows, MinIO for S3-compatible object storage, OpenSearch to index DICOM metadata, Keycloak for login. Includes a built-in PACS, dataset browser, TotalSegmentator and nnU-Net pipelines, and image viewers.	Infrastructure, so no dataset and no accuracy figure. Evidence is real deployment: the RACOON national radiology network, CCE-DART consortium, German Cancer Consortium (DKTK) and NeuroRad project.	Proof that such a platform works in real hospitals. MinIO speaks S3, so our tier logic plugs in directly. OpenSearch already indexes every DICOM tag, which is exactly our feature source. Airflow gives a natural home for our job. Completely free and open source. Send-the-algorithm-to-the-data design suits privacy rules and multi-	Storage is completely flat; everything sits in one class of MinIO bucket with no hot or cold tier. No cost awareness at all. No automatic lifecycle, so nothing ever moves as a scan ages. AI pipelines are purely diagnostic, never used to run the infrastructure. No storage efficiency measurement, and no limitations section.	It owns every ingredient our project needs and connects none of them. A searchable DICOM index, S3 storage, a scheduler and deep learning pipelines all exist, yet a routine X-ray and an urgent trauma CT go into the same

			hospital training.		bucket for ever. The metadata OpenSearch has already indexed is precisely what a tier model needs, and it is used only for search.
P10 - Deng et al. (2026), Project Imaging-X, arXiv:2603.27460	A survey cataloguing over 1,000 open-access medical imaging datasets, indexed by modality, clinical task, body region, annotation type, known limitations and how easily they combine. Proposes a metadata-driven fusion paradigm to merge small datasets into larger ones.	The datasets themselves are the data; there is no experiment and no model. Outputs are an interactive discovery portal, a single unified structured table, and a public GitHub repository.	Directly solves our need for a free and open dataset: one filterable table instead of searching one at a time. Records each dataset's known limitations so we can choose honestly. The fusion idea lets us combine small sets into something trainable. Published 2026, so completely current. Portal and repository are free.	A catalogue, not a method: no algorithm, experiment or result to compare against. Every dataset is diagnostic. None record storage or access behaviour. The landscape is described as fragmented, modest in scale and unevenly spread. Many datasets are converted to PNG or JPEG, destroying the DICOM header. Curation limited by clinical expertise and privacy rules.	Over a thousand free medical imaging datasets exist and not one records who opened which scan and when. That record lives only in hospital PACS audit logs, is protected patient information, and is never released. So the label our model must predict cannot be downloaded and has to be derived from follow-up timestamps instead.

Note: All five papers above are openly available at no cost. Direct links - P6: arxiv.org/abs/2503.20507 | P7: arxiv.org/abs/2603.14583 | P8: arxiv.org/abs/2508.12485 | P9: arxiv.org/abs/2512.09644 | P10: arxiv.org/abs/2603.27460